

Reparación Automática de Software

Test Selection y Mutation Testing

Un poco de terminología inicial

Requisito de Test/Test Requirement (TR): Un elemento de un software que un test debe cubrir.

Esta definición es bastante abstracta, los TR forman conjuntos, y nuestros tests deben cubrir todos los elementos en dicho conjunto. Dada una función $\text{esPar}(x : \text{int})$ podemos definir como TRs a “Es Par” y “Es Impar”, y nuestros tests deberían cubrir ambos requerimientos.

Criterio de Cobertura/Coverage Criterion: un criterio que define cómo generar un conjunto de **Requisitos de Test** para un software particular.

*Se suele hablar de **Cobertura de Criterio X** en lugar de cobertura de requisitos de test, ej: **Cobertura de Ramas**.*

Un poco de terminología inicial

Matriz de Cobertura/Coverage Matrix: Una matriz asociando tests y los requisitos de cobertura que cubren. Para $M[t][r]$, un valor verdadero indica que t cubre r , un valor falso indica que t no cubre r . Se desea que un conjunto de tests (**Test Suite**) tenga una Matriz de Cobertura donde todo requisito es cubierto por al menos un test.

Un poco de terminología inicial

Conjunto minimal de tests/Minimal Test Set: Dado un conjunto T de tests tal que T cubre todos los requisitos TRs, un **Minimal Test Set** va a ser un subconjunto $MT \subseteq T$ tal que MT cubre todos los requisitos TRs y cualquier subconjunto estricto $MT^P \subset MT$ va a fallar en cubrir al menos un TR en TRs.

Conjunto mínimo de tests/Minimum Test Set: Dado un conjunto de Requisitos de Test (**TR**), un conjunto de tests T se dice mínimo/minimum si T cubre todo requisito en TR, y no existe otro conjunto de tests S tal que $|S| < |T|$ y S cubra todo requisito en TR.

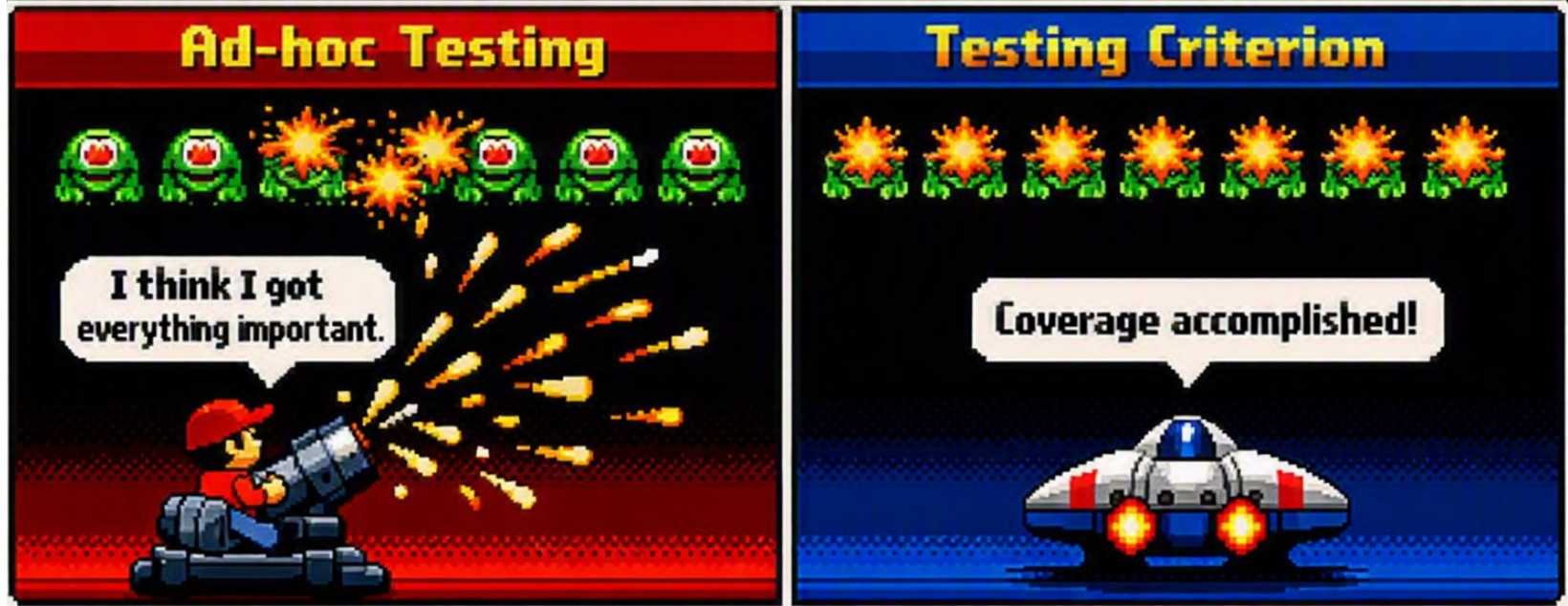
Un poco de terminología inicial

Criteria Subsumption: Un criterio de cobertura C_1 subsume un criterio de cobertura C_2 si y sólo si cada conjunto de tests que satisface C_1 también satisface C_2 .

Test Subsumption: dado dos tests T_1 y T_2 se dice que T_1 subsume a T_2 si cada **TR** cubierto por T_2 es cubierto por T_1 . En el contexto de una matriz de cobertura, T_1 subsume a T_2 si para cada requisito R_i vale

$M[T_2][TR_i]$ implica $M[T_1][TR_i]$

Ventajas de usar criterios de cobertura



Ventajas de usar criterios de cobertura

Dados un **SUT** (Software Under Test) y un conjunto de **Requisitos de Tests TRs**

1. tests = {}
2. mientras $|TRs| > 0$
 - a. generar test t tal que $|tr_cubiertos(TRs, t)| > 0$
 - b. tests $\leftarrow$ tests $\cup \{t\}$
 - c. $TRs \leftarrow TRs \setminus \{tr_cubiertos(TRs, t)\}$
 - d. volver a paso 2
3. retornar tests

Que problemas hay con esto?

Que ventajas hay con esto?

Ventajas de usar criterios de cobertura

Un criterio de cobertura otorga:

1. Claros objetivos a lograr con testing (Test Requirements)
2. Una condición de finalización para escribir tests
3. Una forma de evaluar tests redundantes y calidad de tests

Que problemas hay con un criterio de cobertura?

1. Generar un test que satisfaga uno o más requisitos de tests es un problema indecidible
2. Es posible que haya requisitos de tests que son insatisfacibles

Selección de tests

Dada una test suite **TS** para un SUT (Software Under Test) **P** cubriendo un conjunto de requisitos de test **TRs**.

1. Luego de realizar un cambio en **P**, un problema habitual es seleccionar un subconjunto $\mathbf{TS}_2 \subseteq \mathbf{TS}$ tal que $\mathbf{TS}_2$ cubra todos los requisitos **TRs** afectados por los cambios realizados en **P**.
2. Se desea minimizar **TS** para eliminar tests redundantes.

El problema 2 es relativamente simple de resolver

Minimizando una Test Suite

Dados: un **SUT**, una Test Suite **TS**, y un conjunto de **Requisitos de Tests TRs**

1. Generar un grafo de subsumption ($T_1 \rightarrow T_2 \leftrightarrow T_1$ subsumes T_2)
2. Recolectar nodos del grafo (tests) sin aristas entrantes
3. Retornar los tests recolectados.

Selección de tests

Dada una test suite **TS** para un SUT (Software Under Test) **P** cubriendo un conjunto de requisitos de test **TRs**.

1. Luego de realizar un cambio en **P**, un problema habitual es seleccionar un subconjunto $\mathbf{TS}_2 \subseteq \mathbf{TS}$ tal que $\mathbf{TS}_2$ cubra todos los requisitos **TRs** afectados por los cambios realizados en **P**.
2. Se desea minimizar **TS** para eliminar tests redundantes.

El problema 1 requiere identificar los componentes de **P** asociados con cada test, una opción es utilizar un grafo con control de flujo y determinar para cada test qué partes de este grafo son alcanzados.

Mutation Testing

La idea bajo Mutation Testing es emular defectos y evaluar una test suite en cuanto a su capacidad de detectar estos defectos.

Mutation Testing se basa en 4 partes:

1. Un programa P, correcto, escrito en una gramática G.
2. Un conjunto de operadores de mutación, cada operador especifica una regla de variación/modificación sintáctica de una cadena en G.
3. Mutantes, variaciones de P a partir de una modificación sintáctica generada por un operador de mutación.
4. Evaluar cuantos defectos (mutantes) son detectados por la test suite.

¿Cuales son los Requisitos de Testing?

Mutation Testing

Coberturas asociadas a mutation testing:

1. Mutation Coverage: por cada mutante m se obtiene un requisito de test, “matar al mutante m ” (*)
2. Mutation Operator Coverage: por cada mutador o (operador de mutación) se obtiene un requisito de test, “generar un mutante m mediante o para luego detectar a m ”
3. Mutation Production Coverage: por cada mutador o , y cada elemento mutable por o llamado t , “generar todos los mutantes m generados por o aplicado a t , detectar cada mutante m ”.

(*) Es muy usual el término “matar/kill” en el contexto de mutation testing.

Mutation Score

Dado un programa **P**, un mutante **m** de **P** y un conjunto de test suite **TS** sobre **P** a evaluar, las opciones para **m** son:

1. Al menos un test en **TS** falla para **m** (*), detectando o “matando” a **m**.
2. Ningún test en **TS** falla para **m**, pero **m** contiene un defecto detectable.
3. Ningún test en **TS** falla para **m**, pero **m** no contiene un defecto detectable (**)

(*) Es posible matar un mutante mediante el compilador, el intérprete, o una excepción, estos mutantes suelen considerarse **triviales**.

(**) Un ejemplo son los mutantes equivalentes, el mutante es sintácticamente diferente de **P** pero semánticamente es indistinguible.

Mutation Score

Dado un programa **P**, un mutante **m** de **P** y un conjunto de test suite **TS** sobre **P** a evaluar.

De la misma forma en que se calcula cobertura de sentencias, es decir **sentencias cubiertas / sentencias totales**, podemos calcular mutantes **detectados / mutantes totales** (mutation score).

Strong Mutation Coverage vs Weak Mutation Coverage

Dado un programa **P**, un mutante **m** de **P** y un conjunto de test suite **TS** sobre **P** a evaluar.

Un mutante **m** es “Strongly Killed” si existe al menos un test $t \in \mathbf{TS}$ tal que **t** tiene un output diferente al esperado (obtenido en **P**).

Un mutant **m** es “Weakly Killed” si existe al menos un test $t \in \mathbf{TS}$ tal que el estado luego de la sentencia afectada por **m** es distinta al estado de **P** sobre **t** en la misma sentencia.

Que problemas hay con esto?

Que ventajas hay con esto?

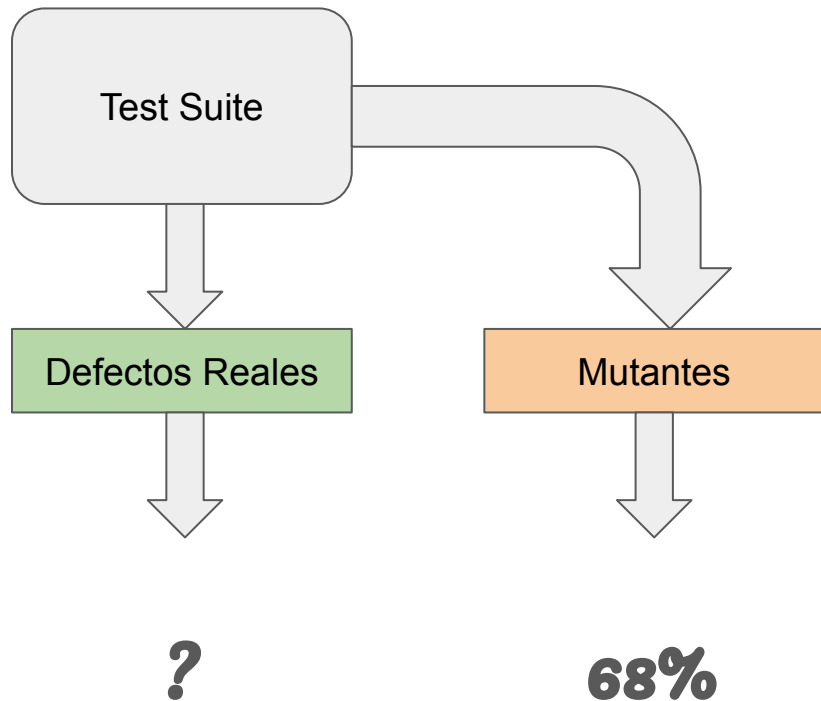
Programador Competente

Si uno intentara insertar posibles defectos en un programa P , para luego evaluar la capacidad de una test suite TS en detectar estos defectos, habría una cantidad infinita de posibles versiones de P .

Sin embargo, si uno tiene en cuenta la competencia de un programador, los defectos que este podría cometer van a crear versiones defectuosas P_D en donde la diferencia (sintáctica) entre P y P_D va a ser “pequeña”.

Esta suposición (empírica) reduce significativamente la cantidad de defectos a considerar para P .

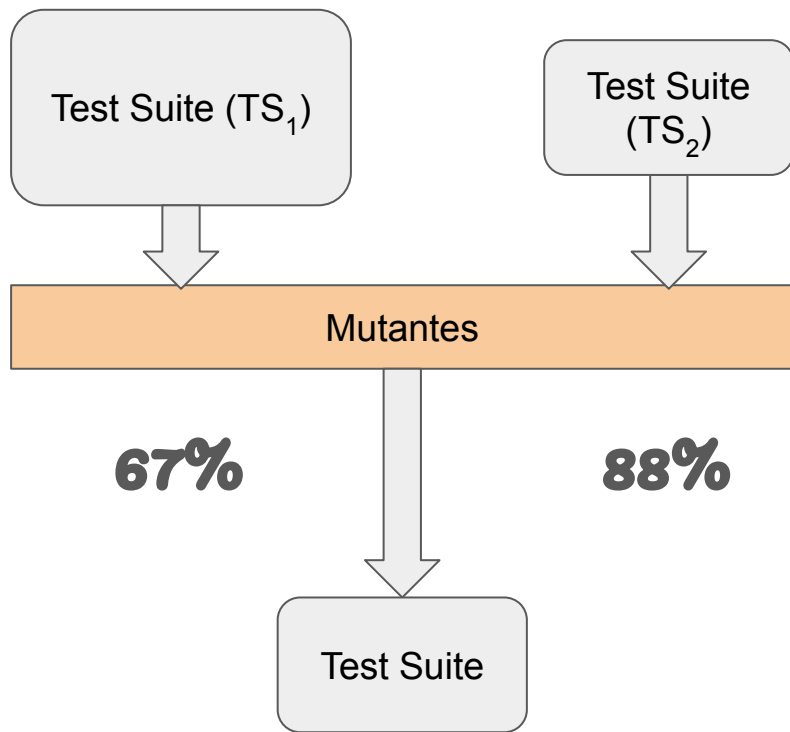
Mutantes en lugar de Defectos Reales



Para evaluar una Test Suite, se suele contar con pocos o ningún defecto real sobre los cuales evaluar.

El uso de mutantes, defectos artificiales, es una práctica común para evaluar la capacidad de una Test Suite para detectar defectos potenciales.

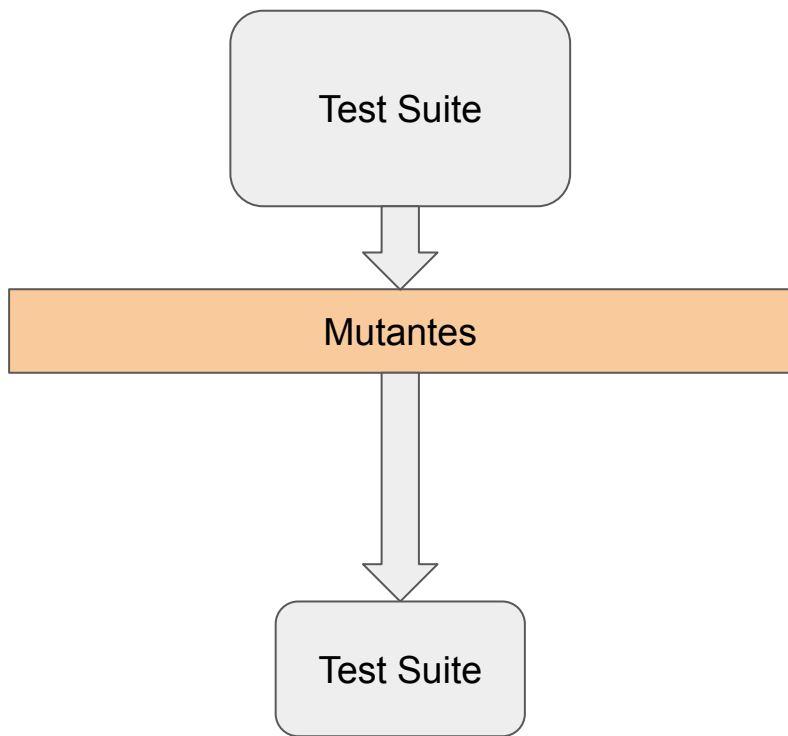
Mutantes en lugar de Defectos Reales



Dadas dos Test Suites TS_1 y TS_2 se pueden utilizar mutantes para seleccionar aquella test suite tal que:

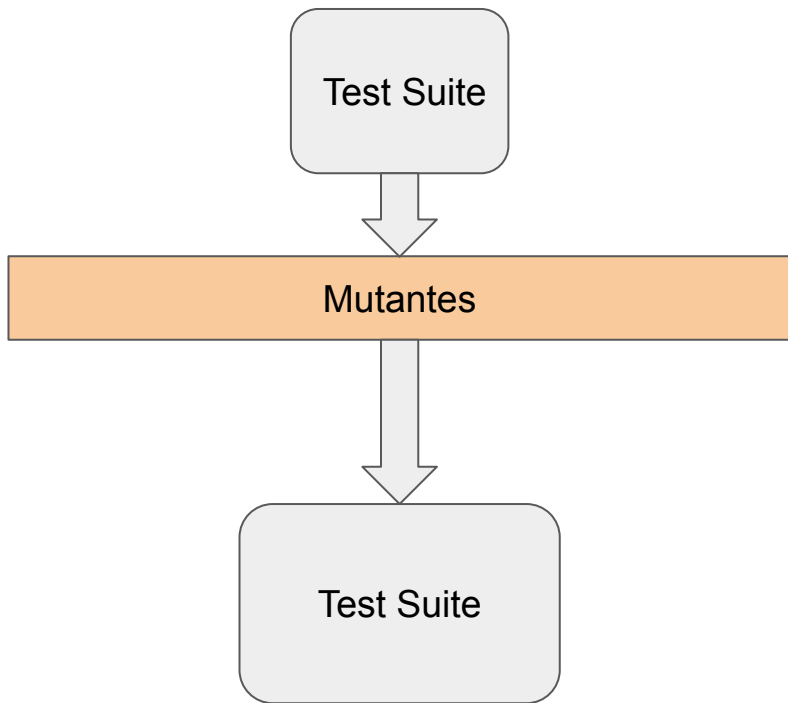
1. Tenga el mismo Mutation Score pero menor tamaño.
2. Tenga mayor Mutation Score.

Mutantes en lugar de Defectos Reales



Dada una test suite **TS**, se puede minimizar eliminando todo test $t \in \mathbf{TS}$ tal que al eliminar t se mantiene el mismo Mutation Score.

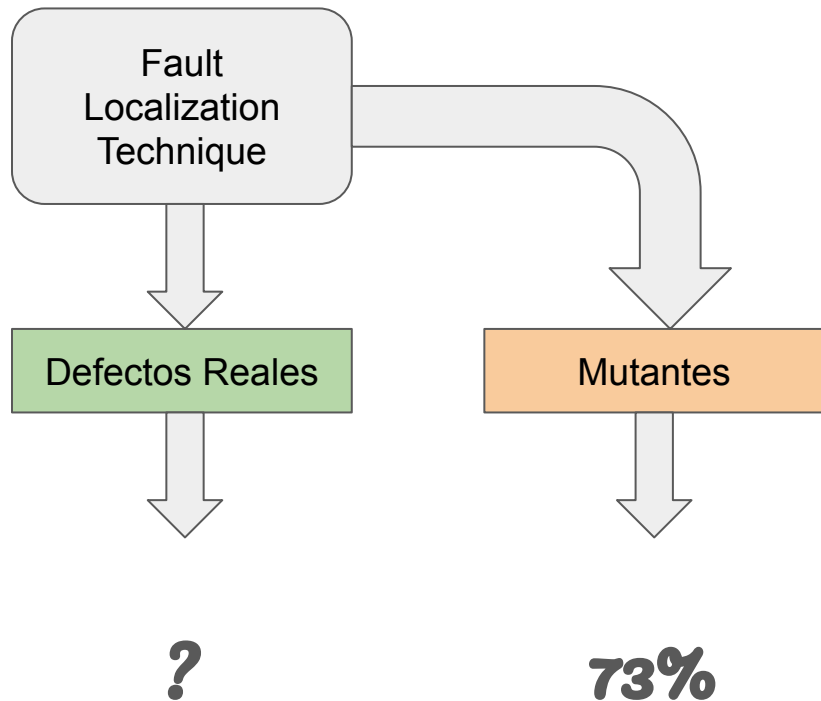
Mutantes en lugar de Defectos Reales



Dada una test suite **TS** inicial, que puede o no estar vacía, se pueden utilizar mutantes para generar nuevos tests automáticamente tal que un nuevo test **t** va a ser agregado a **TS** si y sólo si

$\text{MutationScore}(\mathbf{TS} \cup \{\mathbf{t}\}) > \text{MutationScore}(\mathbf{TS})$.

Mutantes en lugar de Defectos Reales



Para evaluar una técnica de localización de fallas se suele contar con pocos defectos reales sobre los cuales evaluar la técnica.

El uso de mutantes, defectos artificiales, es una práctica común para evaluar la capacidad de una técnica de localización de fallas midiendo su precisión.

Acoplamiento/Coupling Mutantes - Defectos Reales

Un estudio realizado por E. A. Youngs, en su tesis doctoral, llamada “Error-Proneness in Programming” en la Universidad de Carolina del Norte, en 1971* dividió/organizó 1258 errores en categorías de defectos. Estas categorías estaban asociadas a defectos simples, por ejemplo error en asignación/evaluación, error en ramificaciones (condicionales y no condicionales), delimitadores mal ubicados, entre otras.

Este primer estudio presentó la idea de que defectos complejos terminan siendo causados por uno o más defectos simples. Y qué tests capaces de detectar defectos simples suelen ser lo suficientemente sensibles a detectar defectos más complejos.

() La publicación seminal de Mutation Testing fué en 1978 por DeMillo, Lipton, y Sayward, titulada “Hints on Test Data Selection: Help for the Practicing Programmer” en donde se presenta la idea de un Mutation System.*

Acoplamiento/Coupling Mutantes - Defectos Reales

Un segundo estudio realizado por René Just, Darioush Jalali, Laura Inozemtseva, Michael Ernst, Reid Holmes, y Gordon Fraser, en FSE 2014 titulado “Are Mutants a Valid Substitute for Real Faults in Software Testing?” volvió a tratar este tema.

El trabajo tuvo en cuenta casos de estudio reales, provenientes de software de tamaño considerable; se consideró el potencial efecto de cobertura de código en los resultados obtenidos (algo que estudios previos no consideran); se consideraron test suites reales y generadas automáticamente; y se mostraron qué categorías de defectos requerían fortalecer operadores de mutación, cuales requerían operadores nuevos, y cuales no pueden ser modelados/representados por ningún operador de mutación.

Mutation Score - Continuación

Mutation Score es una métrica que fácilmente puede sufrir de los siguientes problemas:

- Sobreestimación, el valor obtenido es más alto debido a mutantes triviales de detectar (por ejemplo: mutantes que no compilan).
- Subestimación, el valor obtenido es más bajo debido a mutantes equivalentes (malo) o difíciles de matar (bueno).

Los mutantes equivalentes disminuyen el mutation score solo por que hay mutantes que son imposibles de detectar (son semánticamente equivalentes al original).

Los mutantes difíciles de matar disminuyen el mutation score, pero en este caso son mutantes útiles de tener, ya que no son semánticamente equivalentes al original, pero son difíciles de detectar.

Toughness - Dureza

Una métrica secundaria es “Cuán difícil es matar a los mutantes”, esta métrica se puede obtener midiendo cuantos tests un mutantes “resiste” antes de “morir”.

Se puede calcular como $1 - \frac{\text{failing tests}}{\text{total tests}}$

No se utiliza como métrica para evaluar tests, sino como métrica para evaluar el mutation score obtenido.

Mutation Subsumption

Al ejecutar una herramienta de Mutation Testing, es posible obtener la Coverage Matrix donde cada defecto es un mutante.

Con esta matriz se puede definir mutation subsumption como:

Dados dos mutantes m_1 y m_2 , m_1 subsume a m_2 si y solo si los tests que detectan a m_2 incluyen todos los que detectan a m_1 .

De la definición anterior se pueden dividir los mutantes en dos categorías:

- Mutantes dominantes: detectados pero no son subsumidos.
- Mutantes redundantes: detectados pero subsumidos.

Por qué los mutantes subsumidos son considerados redundantes?

Operadores (Tomados de μ Java++)

Arithmetic Operators

AODS:	Delete ++ and --
AODU:	Delete unary -
AOIS:	Insert ++ and --
AOIU:	Insert unary -
AORB:	Replace binary arithmetic operators (*, /, %, +, -)
AORS:	Replace ++ with -- and vice versa
AORU:	Replace unary + and -
AOD:	Replace an arithmetic binary operation with one of its operands

Assignment Operators

ASRS:	Replace shortcut assignment operators (+=, -=, *=, /=, %=, &=, =, ^=, <<=, >>=, >>>=) with others
-------	--

Operadores (Tomados de μ Java++)

Conditional Operators

COD: Delete !

COI: Insert !

COR: Replace conditional operators (&&, ||, &, |, ^)

Logical Operators

LOD: Delete ~

LOI: Insert ~

LOR: Replace logical operators (&, |, ^)

Operadores (Tomados de μ Java++)

Relational and Shift Operators

ROR: Replace relational operators ($>$, $\geq$, $<$, $\leq$, $==$, $!=$) and replace predicate with true/false

SOR: Replace shift operators ($<<$, $>>$, $>>>$)

Literal and Constant Mutation Operators (PIT/Major)

CRCR: From constant c generate 1, 0, -1, $-c$, $c + 1$, $c - 1$

IC: Mutate booleans and numeric values depending on type

LVR: Replace numeric literal with 1, 0, -1, string $\rightarrow$ empty, true $\leftrightarrow$ false

Operadores (Tomados de μ Java++)

Class-Level Structural Operators

AMC:	Change access modifiers
IHI:	Hide inherited fields
IHD:	Unhide inherited fields
IOD:	Delete overriding methods
JDC:	Delete default constructor
JID:	Remove field initializers
JSD:	Delete static modifier
JSI:	Insert static modifier

Operadores (Tomados de μ Java++)

Method-Level Structural Operators

EMM:	Replace setter with compatible setter
EOA:	Replace <code>e1 = e2</code> with <code>e1 = (e2).clone()</code>
EOC:	Replace <code>==</code> with <code>equals()</code>
IPC:	Delete <code>super(...)</code> constructor calls
ISI:	Insert <code>super.</code> before accesses
ISD:	Delete <code>super.</code> (<code>super.x</code> $\rightarrow$ <code>x</code> even if invalid)
EAM:	Replace getter with compatible getter
IOP:	Move <code>super.method</code> calls around
JTD:	Delete <code>this</code> (<code>this.x</code> $\rightarrow$ <code>x</code>)
JTI:	Insert <code>this.</code> (<code>var</code> $\rightarrow$ <code>this.var</code>)
OAN:	Replace method call with overriding version (strict type match)

Operadores (Tomados de μ Java++)

Type and Cast Operators

PCC:	Replace cast with cast to direct superclass
PCD:	Delete cast
PCI:	Add cast to parent or child type
PMD:	Change variable/field type to parent type
PPD:	Change method parameter type to parent type

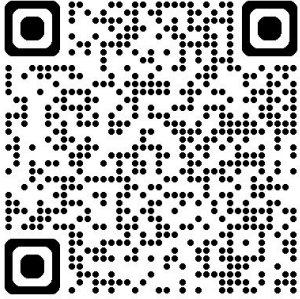
Object Construction Operators

PNC:	Replace new Parent(args) with new Child(args)
------	---

Method Body Operators

OMR:	Replace method body with body of another method with same name
------	--

Frameworks a utilizar



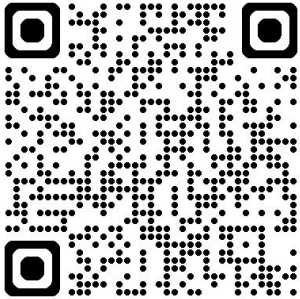
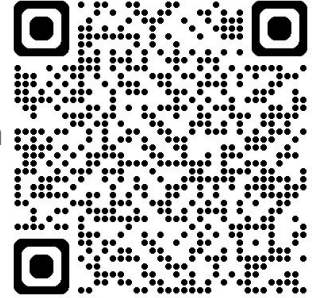
JavaParser (Java)

<https://github.com/javaparser/javaparser>

muJava++ (Java)

<https://github.com/saiema/MuJava>

Solo para experimentar con mutant subsumption



LibCST (Python)

<https://github.com/Instagram/LibCST>

Astroid (Python)

<https://github.com/pylint-dev/astroid>

Ofrece funcionalidad que LibCST no, que son necesarias para mutación

